A Computed Preimage of 20-Round SHA-256 with Standard Padding for a Prescribed Structured Digest

Kameldip Singh Basra
Independent Researcher
kameldipbasra@gmail.com
ORCID: 0009-0001-0977-4614

17 October 2026

Abstract

We report an explicit 55-byte message whose SHA-256 hash, reduced to its first 20 rounds, equals a prescribed structured 256-bit digest. The computation retains the standard initial value, message expansion, round constants, feed-forward, and SHA-256 length padding. The digest was constructed before the search by imposing disclosed relations on the terminal state; the result is therefore a structured-target demonstration, not an arbitrary-target preimage algorithm.

The construction adapts an existing context-shaping framework so that the three message words containing padding depend only on the context and prescribed terminal state. Two word conditions are imposed algebraically; the remaining byte condition selects admissible contexts. Every early-state candidate within such a context inherits valid padding, while the message-schedule equations still have to be satisfied. One recorded run on an NVIDIA A100-SXM4-80GB found the witness after 6,947 context sweeps and 33,474.65 seconds of search time. The log contains 6,944 distinct contexts. A verification release recomputes the result through two forward implementations, checks standard padding and all four schedule residuals, and includes machine-readable evidence.

We also describe the status of an unexecuted extension to the fixed all-ones digest. Its remaining padding filter has 40 constrained bits after one word is eliminated, but a 2^{40} work estimate requires distribution assumptions and excludes the subsequent preimage search. No padded all-ones result, 21-round result, or consequence for full 64-round SHA-256 is claimed.

Keywords: SHA-256; reduced-round preimages; standard padding; context shaping; reproducible verification.

1 Contribution and setting

Let H_R denote SHA-256 with its standard padding and chaining construction, except that each compression call executes rounds $0, \ldots, R-1$. All message words and state words have their actual 32-bit width. This note supplies a message m and digest $h_\star$ such that

$$|m| = 440 \text{ bits}, \qquad H_{20}(m) = h_\star.$$

All 256 output bits match. The initial value is fixed to the standard SHA-256 IV, and feed-forward is retained. The message fits in a single padded block. This distinguishes the result from an unrestricted compression-block preimage, a free-start result, a collision, or a match on only part of the digest.

The target-selection rule is part of the result. A terminal-state family was selected to make padding compatible with the search; its resulting digest was then fixed for the recorded run. This is not evidence of performance on an externally supplied arbitrary digest. The evidence supports a computed example and the algebraic padding construction below. It does not establish a distributional runtime guarantee or a global priority claim.

The underlying table-based context-shaping framework is described in Basra [2]. The additional contribution here is its padded structured-target specialization, the explicit 20-round message, and a verification package. Section 6 separates the proposed padded all-ones extension from both this result and the earlier *unpadded* all-ones compression witness.

2 Definitions and reconstruction identities

All additions and subtractions of words are modulo 2^{32}. Put $M = 2^{32} - 1$ and let $\bar{x} = M \oplus x$. We use the SHA-256 functions and constants of FIPS 180-4 [1]:

$$\begin{aligned}
\Sigma_0(x) &= \mathrm{ROTR}^2(x) \oplus \mathrm{ROTR}^{13}(x) \oplus \mathrm{ROTR}^{22}(x),\\
\Sigma_1(x) &= \mathrm{ROTR}^6(x) \oplus \mathrm{ROTR}^{11}(x) \oplus \mathrm{ROTR}^{25}(x),\\
\sigma_0(x) &= \mathrm{ROTR}^7(x) \oplus \mathrm{ROTR}^{18}(x) \oplus (x \gg 3),\\
\sigma_1(x) &= \mathrm{ROTR}^{17}(x) \oplus \mathrm{ROTR}^{19}(x) \oplus (x \gg 10),\\
\mathrm{Ch}(x,y,z) &= z \oplus (x \,\&\, (y \oplus z)),\\
\mathrm{Maj}(x,y,z) &= (x \,\&\, y) \oplus (x \,\&\, z) \oplus (y \,\&\, z).
\end{aligned}$$

Here K_t is the standard round constant at index t, starting at zero. The standard IV, in hexadecimal, is

```
6a09e667 bb67ae85 3c6ef372 a54ff53a
510e527f 9b05688c 1f83d9ab 5be0cd19
```

Let a_t, e_t be the a, e registers *after* round t. Set $(a_{-1}, a_{-2}, a_{-3}, a_{-4})$ to the first four IV words and $(e_{-1}, e_{-2}, e_{-3}, e_{-4})$ to the last four. Define

$$T_{2,t} = \Sigma_0(a_{t-1}) + \mathrm{Maj}(a_{t-1}, a_{t-2}, a_{t-3}), \tag{1}$$

$$e_t = a_{t-4} + a_t - T_{2,t}, \tag{2}$$

$$W_t = a_t - T_{2,t} - e_{t-4} - \Sigma_1(e_{t-1}) - \mathrm{Ch}(e_{t-1}, e_{t-2}, e_{t-3}) - K_t. \tag{3}$$

These follow directly by eliminating the round temporary $T_{1,t}$. They reconstruct a round-consistent sequence of message words from a proposed state trajectory. They do not by themselves enforce the message schedule.

For 20 rounds, the additional schedule equations are $C_j = 0$ for $j = 0, 1, 2, 3$, where

$$C_j = W_{16+j} - \sigma_1(W_{14+j}) - W_{9+j} - \sigma_0(W_{1+j}) - W_j. \tag{4}$$

Once these hold, $W_0, \ldots, W_{15}$ expand to the reconstructed $W_{16}, \ldots, W_{19}$. The final digest is

$$h = \mathrm{IV} + (a_{19}, a_{18}, a_{17}, a_{16}, e_{19}, e_{18}, e_{17}, e_{16}), \tag{5}$$

with addition componentwise. In particular, $a_{12}, \ldots, a_{19}$ determine the digest: equation (2) gives $e_{16}, \ldots, e_{19}$ using only those eight words.

2.1 Exact padding requirement

A 55-byte message is padded as $m \,\|\, \texttt{80} \,\|\, \mathrm{BE}_{64}(440)$. No zero-fill bytes are inserted between the marker byte and the eight-byte length field in this case. The length field is `00000000000001b8`. Consequently the padding conditions on the input words are

$$W_{13} \bmod 256 = \texttt{80}, \qquad W_{14} = 0, \qquad W_{15} = 440 = \texttt{000001b8}. \tag{6}$$

These are 72 prescribed bits. The upper 24 bits of W_{13} are message data.

3 Padding-compatible structured target

The construction uses the constant $v = \texttt{327853f5}$, which satisfies

$$\Sigma_0(v) = v + 1 = \texttt{327853f6}. \tag{7}$$

Fix the middle-state pattern

$$a_4 = a_5 = a_7 = a_8 = a_9 = v, \qquad a_6 = p, \quad a_{10} = x, \quad a_{11} = y. \tag{8}$$

The early coordinates $a_0, \ldots, a_3$ are the variables explored by the existing chain search. Equations (2) and (7) give $e_8 = e_9 = M$ for every p.

Proposition 1 (Padding belongs to the context)**.** *For a fixed choice of $a_5, \ldots, a_{15}$, the recovered words W_{13}, W_{14}, W_{15} do not depend on $a_0, \ldots, a_3$.*

Proof. Equation (2) makes e_s a function of $a_{s-4}, \ldots, a_s$. Substitution into (3) makes W_t a function of $a_{t-8}, \ldots, a_t$. Thus the three words in question depend only on $a_5, \ldots, a_{15}$. This argument concerns dependency, not statistical independence or the existence of a schedule-consistent early tuple. □

Choose a word t and impose

$$a_{12} = a_{13} = t, \qquad a_{14} = \Sigma_0(t) + 2t, \qquad x = \bar{t}. \tag{9}$$

The words $a_{16}, \ldots, a_{19}$ are selected before the search, and a_{15} is fixed as follows.

Proposition 2 (Two exact padding-word conditions)**.** *Under* (8)–(9)*, setting*

$$a_{15} = 440 + \Sigma_0(a_{14}) + 2t + v + K_{15} \tag{10}$$

makes $W_{15} = 440$ for every p, y. For fixed x, y and terminal words, $W_{14}(p) = W_{14}(0) - p$. Hence $p = W_{14}(0)$ gives $W_{14} = 0$ without changing W_{15}.

Proof. The identities $\mathrm{Maj}(t, y, \bar{t}) = y$ and $\mathrm{Maj}(z, t, t) = t$ yield

$$\begin{aligned} e_{11} &= y - \Sigma_0(x), \\ e_{13} &= v + t - \Sigma_0(t) - y, \\ e_{14} &= x + t = M. \end{aligned}$$

For the third identity, substitute $a_{13} = a_{12} = t$ into $T_{2,14}$ to obtain $T_{2,14} = \Sigma_0(t) + t$. Since $\Sigma_0(\bar{t}) = \overline{\Sigma_0(t)}$, we have $e_{11} + e_{13} = v + t + 1$. Also $T_{2,15} = \Sigma_0(a_{14}) + t$, $\Sigma_1(M) = M$, and $\mathrm{Ch}(M, e_{13}, e_{12}) = e_{13}$. Equation (3) therefore reduces to

$$W_{15} = a_{15} - \Sigma_0(a_{14}) - 2t - v - K_{15},$$

which proves (10). Finally $e_{10} = p + x - 2v - 1$, and the only p-dependent term in W_{14} is $-e_{10}$. The expression for W_{15} contains no p. □

After these two word conditions, the recorded implementation selected y values for which the remaining byte condition in (6) held. A 1/256 acceptance rate is a heuristic for a uniform byte; the construction does not require that assumption for correctness. Once an admissible context is fixed, its early-state candidates all have valid padding. They still have to satisfy the four equations (4) for the *same* trajectory. Padding compatibility is not a completed preimage.

3.1 Published terminal state and chronology

The actual values of $(a_{12}, \ldots, a_{19})$ are

```
da12f9aa da12f9aa e45ab8e9 7cf465f3
2dceac04 06c34cbf a3d15967 5d8ff1ea
```

Equation (5) maps these values to the target in Section 4, without reference to $a_0, \ldots, a_{11}$ or a completed message. The recorded target-generation script used seed 2026; the words above, rather than a particular random-library version, specify the target exactly. The target is present in the recorded run header before the first context sweep. This is an as-run provenance statement, not an external timestamp or independent attestation of the entire search.

4 The computed witness and its verification

The prescribed digest $h_\star$ is

```
c799d8515f3907ec43324031d31ea13e
9d2287d94cf0f4f741989d70463e45b0
```

where the two lines are concatenated. The 55-byte message, likewise concatenated without whitespace, is

```
63823741194daa9bb857c160fe7a02c0
b400ea9d2f4c7f94a33eb2234bd3144e
1c4e9b5d350df3ebf5001d7211a1b83d
0886e96afa6b6b
```

Its standard padded block consists of

```
63823741 194daa9b b857c160 fe7a02c0
b400ea9d 2f4c7f94 a33eb223 4bd3144e
1c4e9b5d 350df3eb f5001d72 11a1b83d
0886e96a fa6b6b80 00000000 000001b8
```

The verification package computes the ordinary forward schedule from these sixteen words, performs exactly twenty rounds from the standard IV, and adds the IV. It obtains $h_\star$. Reconstruction of the recorded trajectory also gives $(C_0, C_1, C_2, C_3) = (0, 0, 0, 0)$. These are separate checks: forward verification does not take the inverse construction's message words beyond W_{15} as input.

The first verifier uses constants derived from integer square and cube roots of primes and an eight-register forward loop. Its 64-round implementation agrees with Python's `hashlib.sha256` on seventeen messages, including padding-boundary and multi-block controls and this witness. The second verifier is the unchanged, standalone `code/verify_r19.py` from the earlier repository snapshot [2]. The two implementations agree at rounds 19, 20, 21, and 64. Neither imports a search kernel. This is implementation cross-checking, not an additional independent search campaign or peer review.

The message does not match $h_\star$ at 19 or 21 rounds. Its full 64-round SHA-256 digest is

```
f28e1032dec89f8ad65b7c3985e41300
c71dc655be08af50b42f81902cc15b7b
```

The complete state trace and adjacent-round digests are supplied in `evidence/verification.json`.

5 Recorded computation and interpretation

The search reused the table-based early-state chain of [2], with the stationary padded contexts of Section 3. The driver swept a_0 across 2^{32} values per context and used three retained-root tables. It then reconstructed reported hits and checked their padding. Three retained roots do not amount to an exhaustive inverse table whenever more roots exist; no completeness claim is made for the searched family.

Quantity	Recorded value
GPU	NVIDIA A100-SXM4-80GB
Search seed	4,024,014,854
Context sweeps / distinct context pairs	6,947 / 6,944
Swept a_0 values, including repeated contexts	29,837,137,805,312
Reported collapsed-candidate count	2,798,787,536
Reconstructed matching blocks	1
Driver elapsed search time	33,474.646 s
Search time in hours	9.2985
Average swept a_0 values per second	8.913×10^8
Logged device-table construction	approximately 6 s

Table 1: One recorded successful run. Work counters are algorithm-specific, not counts of complete SHA-256 evaluations.

The hit was recorded on 16 October 2026 at 21:32:34 UTC, in zero-based context 6946, with $a_0 =$ `5f8abf8e`, $p =$ `3cef771f`, and $y =$ `b53eacdd`. The early state extracted by forward verification agrees with those values. The kernel was a fused Triton implementation; the recorded PyTorch version was 2.4.1+cu124.

The driver timer starts after device-table construction and the table copy used for reconstruction. It includes context generation, sweeps, reconstruction, and loop overhead. The reported rental duration was approximately 9.38 hours. Multiplication by the reported rate of US$1.79 per hour gives about US$16.79. This is a historical cost estimate from the supplied rate and duration, not a measured invoice, an implementation-independent cost, or a forecast for another target.

The public context-counter CSV contains every sweep, including three repeated contexts. Its cumulative collapsed-candidate count is an implementation counter, not a count of distinct independent trials. The driver printed the nominal quantity

$$\lambda_{\text{nominal}} = 2{,}798{,}787{,}536\, 2^{-32} = 0.6516435\ldots.$$

This assigns probability 2^{-32} to each counted residual check. It is not a proved expectation for distinct candidates, nor a probability that the run succeeds. Residual uniformity, independence, and duplicate handling would need to be justified before using such a model predictively. One observed hit does not estimate the mean completion time reliably.

5.1 Bounded pilot comparison

Before the production run, a CPU pilot used a single-root table and four contexts per arm, sweeping 2^{24} early values per context. It recorded 1,707 collapsed candidates in padded contexts and 1,698 in control contexts, with zero complete hits in either arm. The corresponding chain-solution counts were 17,076,155 and 17,070,295 out of 67,108,864 starts per arm. These nearby observations support only a bounded empirical comparison. They do not establish identical distributions or that imposing padding leaves the full success probability unchanged. The pilot's nested low-bit counts and aggregate counters are retained in `evidence/pilot_summary.json`.

6 Status of the all-ones extension

The fixed all-ones digest is the 256-bit value $\texttt{ff}\cdots\texttt{ff}$. The earlier repository contains an unrestricted 20-round compression block for this digest [2]; that block is not a 55-byte padded message. The present release verifies the earlier block as a comparison vector and does not reclassify it as a padded result.

A target-preserving extension is formulated, but has not been executed. For a fixed external digest, the terminal words must come from that digest rather than being chosen to satisfy (9). The structured-target proof making W_{15} constant therefore cannot simply be reused. In the stationary frame, W_{14} still has its affine dependence on p, so one 32-bit padding equation can be eliminated. The remaining padding requirements are one 32-bit word condition and the 8-bit marker condition. Once a compatible context has been found, Proposition 1 places their cost at the context level rather than separately on every early candidate. All four schedule equations remain part of the complete preimage problem.

The heuristic estimate of 2^{40} proposals per admissible context assumes that these remaining 40 bits behave like a uniform joint filter under the chosen sampling policy, with adequate reachability and without problematic repetition. Counting constrained bits alone proves none of those properties. In particular, it neither proves that a suitable context exists for this digest in the selected family nor establishes a measured context-generation rate. The total cost would also include all context sweeps needed to obtain a full preimage, table preparation, and overhead.

Execution status. The padded all-ones route is formulated but unexecuted. No padded all-ones witness, validated search implementation, or measured completion time is established by this release. A future result must supply a concrete message and pass the same target, padding, schedule, and independent forward checks as the structured-target witness.

7 Relation to prior work and limitations

Zaikin [3] reports computed preimages of the 19-round SHA-256 compression function using SAT methods. Basra [2] develops the algebraic context-shaping framework and supplies unrestricted 19- and 20-round compression witnesses. Wain [4] subsequently gives computed 19-round preimages with standard padding, explicitly building on that framework; his separate 20-round repository [5] concerns unrestricted compression blocks. The present verification package replays eleven supplied labelled 19-round padded vectors and one 20-round unpadded vector from Wain, in addition to Basra's unpadded all-ones example. Their own reported performance figures are not re-benchmarked here.

The contribution of this note is a computed *padded* 20-round example for a disclosed structured digest. It does not supersede arbitrary-target claims merely by having the same round count: padding, target selection, available roots, and computational model must all be compared. No claim of first discovery worldwide is made. Theoretical higher-round attacks

also belong to a different comparison: for example, Aoki et al. [6] analyze 43-step SHA-256 preimages at approximately $2^{254.9}$ compression operations. A feasible computed witness and a theoretical attack with enormous complexity answer different questions.

The result supplies no extension to 21 rounds, no general rule for adding rounds, and no attack on full 64-round SHA-256. The algebra establishes padding compatibility under its stated restrictions; the explicit forward computation establishes the witness. Neither establishes general target coverage or an asymptotic runtime law.

Data, code, and assistance disclosure

The accompanying release contains the exact witness, a complete context-counter CSV, numerical run and pilot summaries, a state trace, two forward verifiers, source attribution, and checksums. Verification uses only Python's standard library and no GPU. The release supports independent reproduction of the hash result and consistency checks on the supplied numerical records. It does not include all dependencies of the GPU search kernel and therefore does not by itself reproduce the full search campaign. The mathematical construction and the existence claim are separately checkable from the definitions and witness above.

AI tools assisted with implementation, verification support, and manuscript preparation. Responsibility for the claims and the published materials remains with the author.

A Reproducing the verification

From the top-level directory of the accompanying release, run:

```
python3 verification/verify_release.py --output verification_local.json
```

The script exits with an error on a failed check. It checks standard padding, the 20-round target match, the 19- and 21-round nonmatches, the 64-round `hashlib` controls, every reconstructed schedule word, the four residuals, the stationary and terminal-state relations, the matching context in the run log, and all comparison vectors. Its deterministic output should agree with `evidence/verification.json`:

```
python3 -c "from pathlib import Path; import json; assert \
json.loads(Path('verification_local.json').read_text()) == \
json.loads(Path('evidence/verification.json').read_text())"
```

The unmodified earlier verifier can also check the raw padded block directly:

```
python3 verification/verify_r19.py --rounds 20 \
  --hash c799d8515f3907ec43324031d31ea13e9d2287d94cf0f4f741989d70463e45b0 \
  --words "63823741 194daa9b b857c160 fe7a02c0
           b400ea9d 2f4c7f94 a33eb223 4bd3144e
           1c4e9b5d 350df3eb f5001d72 11a1b83d
           0886e96a fa6b6b80 00000000 000001b8"
```

The raw-block verifier alone does not establish padding; the first command explicitly constructs and checks padding from the message. File integrity can be checked with `sha256sum -c SHA256SUMS`. A checksum checks bytes, not the truth of a scientific interpretation.

References

[1] National Institute of Standards and Technology. *Secure Hash Standard (SHS)*. FIPS PUB 180-4, August 2015. https://doi.org/10.6028/NIST.FIPS.180-4.

[2] K. S. Basra. *Context Shaping for 19- and 20-Round SHA-256 Compression Preimages*. Manuscript and verification repository, 2026. https://github.com/kamb-code/sha256-r19-preimage. Repository snapshot f75b87a620bd; full commit recorded in the accompanying provenance file.

[3] O. Zaikin. Preimage attacks on round-reduced MD5, SHA-1, and SHA-256 using parameterized SAT solver. *Constraints*, published 23 January 2026. https://doi.org/10.1007/s10601-025-09383-0.

[4] S. Wain. *Preimages for 19-step SHA-256 with standard padding*. Public repository, 2026. https://github.com/srw/sha256-19step-preimages-with-padding. Snapshot d9c5abbc2682, accessed 17 October 2026.

[5] S. Wain. *Preimages for 20-step SHA-256 compression function via Zero-a3 Bi-Directional Q0/Q1 Bridge*. Public repository, 2026. https://github.com/srw/sha256-20step-preimages-with-no-padding. Snapshot 0a9cb0c78367, accessed 17 October 2026.

[6] K. Aoki, J. Guo, K. Matusiewicz, Y. Sasaki, and L. Wang. Preimages for Step-Reduced SHA-2. In *ASIACRYPT 2009*, LNCS 5912, pp. 578–597. https://doi.org/10.1007/978-3-642-10366-7_34.